

Corso di Laurea in Ingegneria e Scienze Informatiche

**Generazione automatica di dataset
text-to-SPARQL per fine-tuning di
LLM su knowledge graph
clinico-oncologico**

Tesi di laurea in:
PROGRAMMAZIONE

Relatore

Prof.ssa Antonella Carbonaro

Candidato

Matteo Tonelli

Abstract

Contents

Abstract	iii
1 Introduction	1
2 Background	3
3 Analysis	7
3.1 High-level goals	7
3.2 Constraints	9
3.3 Functional and non-functional requirements	10
3.4 Domain model	12
4 Design	15
4.1 Architecture	15
4.2 Detailed design	18
5 Implementation	23
5.1 Technology stack	23
5.2 Implementation highlights	25
6 Evaluation	29
6.1 Methodology	29
6.2 Quality issues found	31
6.3 Comparison between masking and no-masking generation	35
6.4 Dataset statistics	36
7 Conclusion and Future Work	37
	39
Bibliography	39

CONTENTS

List of Figures

- 3.1 Modello di dominio semplificato: Patient come nodo centrale, con un'entità rappresentativa per ciascuno dei sei tier di priorità 14

LIST OF FIGURES

List of Listings

LIST OF LISTINGS

Chapter 1

Introduction

Structure of the Thesis

Chapter 2

Background

Un knowledge graph è una rappresentazione strutturata della conoscenza in cui le entità del dominio (pazienti, diagnosi, terapie, nel caso del Cancer Virtual Lab) sono modellate come nodi, e le relazioni tra esse come archi etichettati. Questa rappresentazione segue tipicamente il modello RDF (*Resource Description Framework*), in cui ogni fatto è espresso come una tripla soggetto-predicato-oggetto, ad esempio un paziente collegato a una diagnosi tramite la relazione `has_diagnosis`.

SPARQL è il linguaggio di interrogazione standard per dati RDF, analogo per ruolo a SQL nel contesto delle basi di dati relazionali. Una query SPARQL specifica un pattern di triple da individuare nel grafo, con la possibilità di applicare filtri sui valori e vincoli sulla struttura delle relazioni attraversate.

Il problema del text-to-SPARQL si inserisce in una famiglia più ampia di compiti noti in letteratura come semantic parsing, in cui un sistema deve tradurre un enunciato in linguaggio naturale in una rappresentazione formale eseguibile. Nel caso specifico del text-to-SQL e del text-to-SPARQL, la rappresentazione target è un linguaggio di interrogazione strutturato, e la difficoltà principale risiede nella necessità di colmare il divario tra la libertà espressiva del linguaggio naturale e la rigidità sintattica e semantica del linguaggio target. Un modello deve non solo comprendere l'intento della domanda, ma anche conoscere con precisione lo schema sottostante, ovvero i nomi delle entità, delle proprietà e delle relazioni disponibili, poiché un errore anche minimo nella scelta di un predicato produce una query sintatticamente valida ma semanticamente errata.

Nel dominio specifico del text-to-SPARQL su knowledge graph clinici, questa difficoltà è accentuata da due fattori. In primo luogo, lo schema è spesso altamente specialistico e non presente nella conoscenza pregressa di un modello linguistico generalista, che non ha modo di conoscere a priori l'esistenza di una proprietà come `has_ECOG_status` o di un'entità come *Pathological TNM*. In secondo luogo, la scarsità di dati annotati è un problema ricorrente in ambito clinico, dove la produzione di coppie domanda-query richiede competenze di dominio che non tutti gli annotatori possiedono.

Diversi lavori in letteratura hanno affrontato il problema della costruzione di dataset per il text-to-SPARQL, adottando strategie differenti per generare la componente in linguaggio naturale a partire da query strutturate. LC-QuAD, nelle sue due versioni [TMDL17, DBAL19], costruisce le domande a partire da template applicati al grafo, genera una domanda intermedia in linguaggio quasi naturale (NNQT, *Normalized Natural Question Template*) e infine affida a annotatori umani, tramite piattaforme di crowdsourcing, la parafrasi finale in linguaggio naturale fluente, producendo una singola parafrasi per ciascuna domanda. MK-SQuIT [SCKS20] adotta invece un approccio interamente automatico, generando sia la query sia la sua controparte testuale tramite una grammatica formale, senza alcuna fase di parafrasi successiva: la variabilità linguistica del dataset dipende quindi unicamente dalla ricchezza della grammatica stessa. Un lavoro più recente di IBM Research, condotto da Bulle Labate e colleghi [LdCF25], applica invece un modello linguistico di grandi dimensioni per rigenerare le parafrasi delle domande già presenti in LC-QuAD, validando la qualità del risultato tramite un secondo modello linguistico nel ruolo di giudice, secondo cinque criteri valutati su una scala da uno a cinque.

Questi tre lavori delineano uno spettro di possibili approcci: da un estremo interamente manuale nella fase di parafrasi, a un estremo interamente automatico privo di parafrasi, fino a un approccio ibrido in cui l'automazione tramite modello linguistico sostituisce l'annotatore umano pur mantenendo una fase di parafrasi esplicita. L'utilizzo di modelli linguistici per l'augmentation testuale non è tuttavia privo di rischi, in particolare quando applicato a domini con vocabolario controllato: un modello può introdurre sinonimi lessicalmente plausibili ma semanticamente non equivalenti al termine originale, alterando il significato clinico

di un valore senza che questo risulti evidente da un controllo superficiale del testo prodotto.

Il lavoro descritto in questa tesi si colloca in prossimità dell'estremo automatico dello spettro appena descritto, condividendo con MK-SQuIT l'assenza di annotazione umana nella fase di parafrasi, ma introducendo, a differenza di quest'ultimo, una fase di validazione programmatica multilivello sull'output del modello linguistico, distinta sia dall'approccio di IBM Research, che affida la validazione a un secondo modello linguistico, sia dall'assenza di validazione esplicita in MK-SQuIT.

Le tecniche di augmentation testuale basate su modelli linguistici di grandi dimensioni si sono affermate come alternativa alla parafrasi manuale in numerosi compiti di elaborazione del linguaggio naturale, sfruttando la capacità di questi modelli di produrre riformulazioni fluenti e grammaticalmente corrette a partire da un testo sorgente. Rispetto a tecniche più risalenti, come la sostituzione di sinonimi tramite risorse lessicali statiche o la back-translation attraverso un sistema di traduzione automatica, un modello linguistico generativo consente una variazione più ampia nella struttura sintattica e nel registro della frase prodotta, non limitandosi alla sola sostituzione lessicale.

Questa flessibilità comporta tuttavia un rischio strutturale, distinto dal semplice errore casuale: un modello linguistico generalista non possiede, per costruzione, conoscenza affidabile di vocabolari controllati specifici di un dominio, quali codici di classificazione clinica, terminologie normalizzate, o distinzioni concettuali che a un lettore non esperto del dominio possono apparire equivalenti pur non essendolo. Nel richiedere una parafrasi o un sinonimo per un termine di questo tipo, il modello può produrre un'alternativa lessicalmente plausibile ma semanticamente non equivalente, senza segnalare in alcun modo l'incertezza sottostante. Questo comportamento si manifesta tipicamente in due forme distinte. La prima è la sostituzione di un termine tecnico con un termine di registro comune tratto da un dominio concettualmente adiacente ma distinto, che altera il contesto interpretativo del valore pur mantenendone una plausibilità superficiale. La seconda, più marcata, è l'allucinazione vera e propria, in cui il modello associa a un codice o a un identificativo privo di significato lessicale intrinseco un'interpretazione testuale inventata, spesso non verificabile a partire dal solo contesto della frase prodotta.

La rilevanza di questi rischi per un dataset destinato al fine-tuning non si limita alla qualità percepita del singolo esempio generato: un modello allenato su dati contenenti sostituzioni sistematicamente errate rischia di apprendere associazioni scorrette tra forma linguistica e significato clinico, propagando l'errore oltre il singolo esempio da cui ha avuto origine. Per questa ragione, l'adozione di tecniche di augmentation basate su modelli linguistici richiede, più che nei casi in cui il vocabolario di dominio non sia critico, una fase di validazione esplicita dell'output prodotto, distinta dalla sola verifica di correttezza grammaticale o di fluidità superficiale della frase.

Nel capitolo dedicato alla valutazione del dataset prodotto in questo lavoro (chapter 6) sono documentati casi concreti riconducibili a entrambe le forme di errore appena descritte, insieme alla metodologia adottata per individuarli e alle correzioni apportate al sistema di generazione delle parafrasi.

In sintesi, il presente lavoro si colloca in un punto specifico dello spettro delineato dai tre approcci discussi: automatico nella generazione della componente strutturata e nella parafrasi, come MK-SQuIT, ma dotato di un meccanismo di validazione esplicito e multilivello che né MK-SQuIT né, in forma diversa, IBM Research adottano nella medesima misura. I capitoli successivi descrivono nel dettaglio l'architettura adottata (chapter 4), la sua implementazione (chapter 5), e i risultati della validazione condotta sul dataset prodotto (chapter 6).

Chapter 3

Analysis

3.1 High-level goals

L’obiettivo primario del progetto è la costruzione di un dataset annotato per il fine-tuning di un modello linguistico di dimensioni contenute, destinato alla traduzione di domande in linguaggio naturale in query SPARQL sul knowledge graph del Cancer Virtual Lab. Da questo obiettivo generale derivano cinque obiettivi più specifici, che hanno guidato le scelte progettuali descritte nei capitoli successivi e a cui si farà riferimento nel seguito del documento.

Il primo obiettivo è la copertura di un sottoinsieme rappresentativo dello schema del knowledge graph, comprendente quarantacinque entità selezionate e organizzate in sei livelli di priorità, con una distribuzione del volume di dati deliberatamente non uniforme tra i livelli, orientata verso le entità cliniche di uso più frequente nel contesto applicativo. Questa scelta nasce dalla considerazione che un dataset costruito con copertura uniforme su tutto lo schema disperderebbe risorse di generazione su entità raramente interrogate nella pratica clinica, a scapito della profondità di copertura sulle entità realmente centrali per l’uso previsto del sistema. Il sistema di generazione è inoltre progettato per essere estendibile a porzioni ulteriori dello schema, non ancora coperte nella versione attuale del dataset, poiché il sottoinsieme di quarantacinque entità rappresenta una scelta di ambito per questo lavoro, non un limite strutturale del sistema di generazione stesso.

Il secondo obiettivo è la produzione di varietà linguistica autentica nelle do-

mande generate, attraverso dieci registri stilistici distinti, senza che questa varietà comprometta la correttezza semantica del testo prodotto rispetto alla query sottostante. La motivazione di questo obiettivo risiede nel rischio, discusso nel capitolo di background, che un dataset di addestramento privo di varietà linguistica reale produca un modello capace di riconoscere unicamente formulazioni molto simili a quelle viste in fase di addestramento, con conseguente scarsa capacità di generalizzazione a domande poste con un fraseggio diverso da un utente reale. Al tempo stesso, la varietà non è un obiettivo fine a se stesso: un sistema che massimizzasse la variazione linguistica senza vincoli produrrebbe inevitabilmente, in una frazione dei casi, varianti che alterano il significato della domanda originale, un rischio discusso nel dettaglio nel capitolo di valutazione. L'obiettivo, pertanto, è definito esplicitamente come un compromesso tra questi due requisiti contrastanti, non come una massimizzazione indipendente di uno dei due.

Il terzo obiettivo è la tracciabilità completa: ogni frase generata deve essere riconducibile in modo univoco e verificabile alla query SPARQL che la genera, condizione necessaria per un utilizzo affidabile come dato di addestramento supervisionato. Questo obiettivo, apparentemente elementare, ha conseguenze dirette sulle scelte di design descritte nel capitolo successivo: la necessità di preservare un collegamento verificabile tra ciascuna variante testuale e la query originale è ciò che ha motivato l'introduzione del meccanismo di mascheramento dei valori, poiché una parafrasi generata liberamente, in assenza di questo vincolo, non offre alcuna garanzia che i valori numerici, gli identificativi, o i codici presenti nella query compaiano ancora, nella loro forma esatta o in una forma alternativa verificabile, nel testo prodotto.

Il quarto obiettivo è la produzione di un volume di dati sufficiente per un fine-tuning efficiente in termini di risorse computazionali, applicato a un modello linguistico di dimensioni contenute. Le tecniche specifiche adottate per il fine-tuning, e la motivazione della scelta di un modello di dimensioni contenute rispetto ad alternative di scala maggiore, sono descritte nel capitolo dedicato all'implementazione; a livello di obiettivo di alto livello, ciò che rileva è che il volume di dati necessario per un addestramento efficace dipende dalla tecnica di fine-tuning impiegata, e che questo obiettivo ha guidato, insieme al primo, le decisioni relative al numero di varianti prodotte per ciascun record sorgente.

Il quinto obiettivo è la validazione sistematica della correttezza semantica delle varianti prodotte, attraverso un processo di controllo qualità in grado di individuare alterazioni non intenzionali del significato clinico introdotte dal meccanismo di generazione linguistica. Questo obiettivo è stato formulato esplicitamente come distinto dal terzo, sebbene entrambi riguardino la correttezza del dataset prodotto: la tracciabilità garantisce che un valore compaia nel testo generato, ma non garantisce che il testo circostante quel valore non ne alteri l'interpretazione clinica, un rischio discusso nel dettaglio nel capitolo di background e documentato con casi concreti nel capitolo di valutazione. Un dataset che soddisfacesse il terzo obiettivo senza soddisfare il quinto potrebbe pertanto contenere valori formalmente presenti ma inseriti in un contesto linguistico fuorviante, una condizione che nessun controllo di sola tracciabilità sarebbe in grado di rilevare.

3.2 Constraints

Il progetto è stato condotto sotto un insieme di vincoli tecnici e organizzativi che ne hanno influenzato le scelte architetturelle, descritte nel dettaglio nei capitoli successivi. A differenza degli obiettivi discussi nella sezione precedente, che rappresentano condizioni desiderabili verso cui il sistema è stato orientato, i vincoli discussi in questa sezione rappresentano condizioni esterne, non modificabili nell'ambito del progetto, entro cui il sistema ha dovuto necessariamente operare.

Sul piano tecnico, la fase di augmentation testuale si basa su un modello linguistico eseguito localmente, scelta motivata dall'assenza di dipendenza da servizi esterni a pagamento e dal controllo diretto sui tempi di esecuzione, a fronte di una minore capacità del modello rispetto a soluzioni commerciali di dimensioni maggiori. Questo vincolo ha conseguenze dirette sul design del sistema di generazione delle varianti, discusso nel capitolo successivo: un modello di dimensioni contenute, eseguito senza l'assistenza di infrastrutture di calcolo dedicate durante le fasi iniziali del progetto, produce con maggiore frequenza output non conformi ai vincoli richiesti rispetto a un modello di scala superiore, rendendo necessario un meccanismo di validazione e ripetizione dei tentativi di generazione più articolato di quanto sarebbe stato richiesto con un modello più capace.

L'infrastruttura di calcolo disponibile è inoltre variata nel corso del progetto,

introducendo un vincolo di natura temporale distinto da quello appena discusso. Alcune fasi di sviluppo sono state condotte in assenza di accelerazione grafica dedicata, con tempi di generazione dell'ordine dei minuti per singola frase, un ordine di grandezza che ha reso impraticabile la generazione dell'intero dataset in questa configurazione e ha conseguentemente vincolato le prime fasi di sviluppo a operare su campioni ridotti, a scopo di verifica del sistema piuttosto che di produzione del dataset finale. In una fase successiva si è resa disponibile una scheda grafica di fascia consumer con memoria video limitata, sufficiente per la sola fase di augmentation ma non per il fine-tuning del modello, un limite che ha reso necessario distinguere, nella pianificazione del progetto, l'infrastruttura destinata alla generazione del dataset da quella destinata all'addestramento del modello. La fase di fine-tuning vera e propria richiede infine l'accesso a un cluster di calcolo condiviso, con tempi di attesa in coda non prevedibili a priori, un vincolo che ha imposto la progettazione di meccanismi di ripresa automatica dell'esecuzione, descritti nel capitolo di design, poiché un'attesa non prevedibile rende inaffidabile qualunque pianificazione che presupponga il completamento ininterrotto di un'esecuzione di lunga durata. Le scelte tecnologiche specifiche adottate per ciascuna fase sono descritte nel capitolo dedicato all'implementazione.

Un vincolo di natura organizzativa, distinto dai vincoli tecnici discussi finora, è rappresentato dalla scadenza dell'appello di laurea. Questo vincolo ha imposto una pianificazione del lavoro orientata a garantire la disponibilità di un dataset completo e validato con margine sufficiente rispetto alla fase di fine-tuning e stesura finale del documento, e ha influenzato in particolare l'ordine in cui i diversi problemi di qualità, discussi nel capitolo di valutazione, sono stati affrontati: la priorità è stata assegnata alla correzione dei problemi con maggiore impatto quantitativo sul dataset, quali l'allucinazione di codici clinici e la conflazione tra sesso anagrafico e identità di genere, rispetto a problemi di portata più limitata individuati nelle fasi successive del progetto.

3.3 Functional and non-functional requirements

I requisiti funzionali del sistema descrivono cosa il sistema deve produrre, indipendentemente da come tale produzione venga realizzata. Il sistema deve generare

coppie costituite da una query SPARQL sintatticamente valida e da un prompt in linguaggio naturale corrispondente, a partire dallo schema del knowledge graph; questo requisito costituisce il nucleo minimo indispensabile del sistema, poiché senza di esso non esisterebbe alcun dato su cui applicare i requisiti successivi. Il sistema deve produrre, per ciascuna coppia generata, un numero variabile di varianti parafrasate del medesimo prompt, preservando invariata la query associata; la variabilità del numero di varianti, anziché un numero fisso uguale per ogni coppia, riflette la considerazione, discussa nel capitolo di design, che record di complessità diversa offrono un margine di variazione linguistica naturale diverso, e un numero fisso applicato indistintamente rischierebbe di essere insufficiente per i record più complessi o ridondante per i più semplici.

Il sistema deve inoltre garantire la preservazione esatta dei valori letterali presenti nella query, quali identificativi di pazienti, codici di classificazione clinica, e date, in ciascuna variante parafrasata prodotta, oppure, quando ammesso dal sistema di sinonimi descritto nel capitolo di design, la presenza di una forma alternativa verificabile del medesimo valore. Questo requisito discende direttamente dal terzo obiettivo di alto livello, la tracciabilità, e ne costituisce la condizione di verificabilità concreta: un dataset in cui questo requisito non fosse soddisfatto non permetterebbe di stabilire con certezza se una variante testuale corrisponda ancora alla query da cui è stata generata. Il sistema deve infine supportare la generazione di query di complessità variabile, dal caso di un singolo vincolo su una singola proprietà fino a query che attraversano più relazioni e combinano più vincoli sullo stesso nodo, un requisito che discende dal primo obiettivo di alto livello e che ha motivato l'introduzione del sistema di parametri configurabili per tier descritto nel capitolo successivo.

I requisiti non funzionali riguardano invece proprietà trasversali del sistema, non direttamente osservabili nel singolo output prodotto, ma nel comportamento del sistema considerato nel suo complesso. Il sistema deve mantenere un throughput di generazione compatibile con i tempi disponibili per il progetto, tenendo conto delle differenti infrastrutture di calcolo utilizzate nelle diverse fasi; questo requisito è in tensione diretta con il vincolo relativo all'infrastruttura di calcolo discusso nella sezione precedente, ed è stato uno dei fattori che ha motivato l'adozione di un meccanismo di generazione batch eseguibile senza supervisione, in grado di

proseguire per periodi prolungati senza intervento manuale.

Il sistema deve inoltre garantire un tasso di accettazione delle generazioni, inteso come rapporto tra varianti prodotte con successo e tentativi complessivi, sufficientemente elevato da rendere il processo economicamente sostenibile in termini di tempo di calcolo: un tasso di accettazione eccessivamente basso implicherebbe che una frazione rilevante del tempo di calcolo disponibile, esso stesso vincolato come discusso nella sezione precedente, venga impiegata in tentativi di generazione scartati anziché in output utilizzabile. Il sistema deve essere inoltre riproducibile in caso di interruzione, ovvero in grado di riprendere una generazione interrotta senza ripetere il lavoro già completato, un requisito motivato direttamente dall'esperienza, discussa nel capitolo di implementazione, di interruzioni ripetute dovute sia a vincoli di tempo sull'infrastruttura di calcolo condivisa sia a errori occorsi durante l'esecuzione. Infine, il sistema deve essere robusto rispetto a un rischio specifico del dominio applicativo, ovvero la possibilità che il meccanismo di variazione linguistica alteri, senza segnalarlo, il significato di un valore clinico controllato; questo requisito discende direttamente dal quinto obiettivo di alto livello, e la sua violazione, quando verificatasi nel corso del progetto, è documentata con casi concreti nel capitolo di valutazione.

3.4 Domain model

Il modello di dominio adottato in questo progetto non coincide con l'intero schema del knowledge graph del Cancer Virtual Lab, ma con un sottoinsieme di quarantacinque entità, selezionate e organizzate in sei livelli di priorità denominati Tier 0 e Tier da A a E. Questa organizzazione a livelli non riflette una proprietà intrinseca dello schema, bensì una decisione progettuale: il sistema di generazione è stato costruito in modo scalabile, e la scelta di quali entità includere e con quale peso relativo è stata guidata dalla necessità di produrre un volume di dati sbilanciato verso le entità cliniche più frequentemente interrogate nel contesto applicativo del Cancer Virtual Lab, piuttosto che verso una copertura uniforme dell'intero schema. Questa scelta distingue il modello di dominio qui descritto da quello tipico di un progetto costruito da zero, in cui l'analisi del dominio ha il compito di far emergere per la prima volta le entità rilevanti: in questo caso il dominio, ovvero l'ontologia

clinica del Cancer Virtual Lab, preesisteva al progetto, e il lavoro di modellazione qui descritto consiste nella selezione e prioritizzazione di un sottoinsieme di tale ontologia, non nella sua definizione originaria.

La struttura a sei livelli è illustrata in fig. 3.1, che mostra il nodo Patient come centro dello schema e, per ciascun tier, un'entità rappresentativa collegata attraverso il numero di passaggi relazionali corrispondente al livello di appartenenza. Il Tier 0 comprende la sola entità Patient, nodo centrale dello schema e punto di partenza di ogni relazione considerata nel sistema. Il Tier A comprende undici entità cliniche direttamente collegate al paziente, quali Diagnosis e Therapy, rappresentanti le informazioni cliniche di uso più frequente nel contesto applicativo e per questo motivo collocate al livello di priorità immediatamente successivo al paziente stesso. Il Tier B comprende sei entità raggiungibili con due passaggi relazionali dal paziente, tipicamente informazioni accessorie a un'entità di Tier A, quali il dettaglio di una singola somministrazione nell'ambito di una terapia. I Tier C, D ed E comprendono rispettivamente tre, undici e tredici entità più periferiche, relative a sostanze attive ed eventi avversi, sintomi e fattori di stile di vita, e classificazioni astratte quali le componenti della stadiazione Tumor, Node, Metastasis (TNM); queste ultime, pur numericamente rilevanti nello schema complessivo, rappresentano informazioni consultate con minore frequenza nel contesto applicativo rispetto alle entità dei tier superiori, e sono pertanto collocate ai livelli di priorità più bassi. Il numero di record generati per ciascun tier decresce progressivamente da Tier 0 a Tier E, riflettendo direttamente questa priorità decrescente nella quantità di dati prodotti per ciascun livello.

Per ciascuna entità, la complessità delle query generabili è controllata da quattro parametri indipendenti: il numero di relazioni attraversate nel grafo a partire dal nodo radice, il numero di percorsi alternativi combinati nella medesima query tramite clausole di disgiunzione, il numero di proprietà vincolate su un medesimo nodo, e il numero di valori ammessi per ciascun vincolo. Questi quattro parametri sono stati scelti perché corrispondono a quattro dimensioni distinte lungo cui una query SPARQL può variare in complessità, indipendentemente l'una dall'altra: due query possono attraversare lo stesso numero di relazioni ma differire nel numero di proprietà vincolate su un singolo nodo, o viceversa, ed entrambe le variazioni contribuiscono alla complessità percepita della query ma in modo qualitativamente

diverso. La combinazione di questi quattro parametri, differenziata per ciascun tier secondo un intervallo minimo e massimo specifico, determina la distribuzione di complessità delle query nel dataset finale, ed è descritta nel dettaglio, insieme al meccanismo con cui è stata successivamente estesa nel corso del progetto, nel capitolo dedicato al design del sistema di generazione.

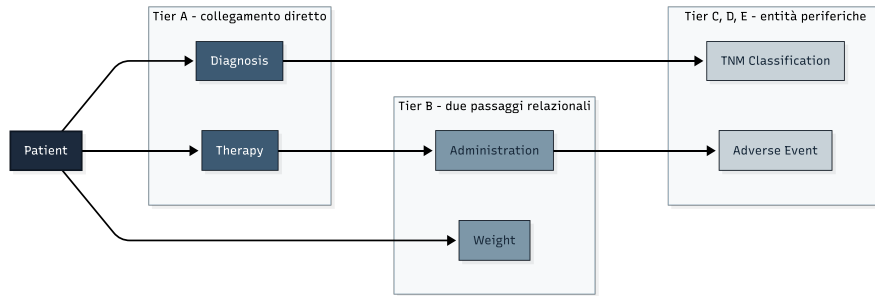


Figure 3.1: Modello di dominio semplificato: Patient come nodo centrale, con un'entità rappresentativa per ciascuno dei sei tier di priorità

Chapter 4

Design

4.1 Architecture

L’architettura del sistema è organizzata in due stadi sequenziali e disaccoppiati, comunicanti attraverso un formato dati intermedio esplicito: un primo stadio di generazione strutturata, responsabile della produzione di coppie prompt-query sintatticamente e semanticamente corrette a partire dallo schema del knowledge graph, e un secondo stadio di augmentation linguistica, responsabile dell’espansione di ciascuna coppia in un numero variabile di varianti parafrasate in linguaggio naturale.

Il primo stadio interroga direttamente l’endpoint SPARQL del Cancer Virtual Lab, esposto tramite un’interfaccia web e containerizzato in un ambiente Docker, per verificare l’esistenza e il numero di istanze corrispondenti a ciascuna combinazione di entità, relazioni e valori prima di includerla nel dataset. Questa verifica preventiva garantisce che ogni query generata corrisponda a dati realmente presenti nel knowledge graph, evitando la produzione di coppie prompt-query sintatticamente corrette ma prive di corrispondenza nei dati reali. Il primo stadio produce quindi, per ciascuna combinazione valida, tre elementi: il prompt in forma template, privo di variazione linguistica; la query SPARQL corrispondente, completa e direttamente eseguibile; e la posizione esatta, espressa come intervallo di caratteri all’interno del testo del prompt, di ciascun valore letterale che compare anche nella query.

Questa terza informazione, che chiameremo intervallo di valore, è l'elemento cardine dell'interfaccia tra i due stadi. Senza di essa, il secondo stadio non avrebbe modo di distinguere, all'interno del testo del prompt, quali porzioni rappresentino valori critici da preservare esattamente durante la parafrasi e quali rappresentino invece testo di connessione liberamente riformulabile. Concretamente, ciascun record nel formato di interscambio tra i due stadi assume una struttura di questo tipo: un identificativo univoco, il testo del prompt, la query SPARQL, e un elenco di intervalli, ciascuno costituito da una posizione di inizio e una di fine all'interno del testo del prompt. Il secondo stadio utilizza questi intervalli sia per il mascheramento dei valori prima dell'invocazione del modello linguistico, sia per la validazione successiva, verificando che ciascun valore originale, o una sua forma alternativa ammessa, compaia effettivamente nel testo prodotto.

La separazione tra i due stadi non è unicamente una comodità organizzativa, ma riflette una distinzione sostanziale nel tipo di garanzia offerta da ciascuno, e ha avuto conseguenze pratiche dirette nel corso dello sviluppo del sistema. Nel primo stadio, la correttezza della query prodotta è garantita per costruzione: ogni query deriva da una combinazione verificata di entità, relazioni e valori realmente presenti nel knowledge graph, e non da un processo generativo soggetto a errore. Nel secondo stadio, al contrario, la correttezza del testo prodotto non può essere garantita a priori, poiché la generazione è delegata a un modello linguistico; essa viene pertanto verificata a posteriori, attraverso un meccanismo di validazione descritto nel capitolo dedicato al design di dettaglio. Questa distinzione ha permesso, nel corso dello sviluppo, di isolare e correggere problemi specifici di uno stadio senza dover intervenire sull'altro: un difetto individuato nel sistema di generazione dei sinonimi del secondo stadio, ad esempio, è stato corretto e la sola fase di augmentation è stata rieseguita sui record interessati, senza necessità di rigenerare le query strutturate prodotte dal primo stadio, che restavano corrette.

L'architettura del secondo stadio prevede inoltre due modalità di esecuzione alternative, condividendo la stessa validazione a valle ma differenti nel trattamento dei valori durante la generazione. Nella modalità principale, i valori vengono mascherati con segnaposti opachi prima dell'invocazione del modello linguistico, e reinseriti successivamente. Una modalità alternativa, introdotta per un confronto sperimentale documentato nel capitolo di valutazione, omette il maschera-

mento e sottopone al modello il testo con i valori originali visibili, accompagnati da suggerimenti di forma alternativa come indicazioni facoltative anziché come sostituzioni post-hoc. Questa seconda modalità non costituisce un ramo architetturale separato, ma una configurazione alternativa dello stesso stadio, condividendo l'infrastruttura di validazione, e la sola differenza risiede nel trattamento dei valori durante la fase di generazione.

Infine, l'architettura del primo stadio è stata progettata per supportare un'estensione incrementale del dataset senza richiedere la rigenerazione delle porzioni già prodotte. Questa proprietà architettuale, motivata dalla necessità emersa nel corso del progetto di ampliare la copertura del dataset senza duplicare il lavoro già svolto, è realizzata attraverso un secondo insieme di parametri di generazione, distinto da quello principale e descritto nel dettaglio nel capitolo successivo, che vincola strutturalmente i nuovi record a non sovrapporsi a quelli già esistenti.

Un'ulteriore proprietà architettuale, trasversale a entrambi gli stadi, è la capacità di ripresa dell'esecuzione. Poiché la generazione dell'intero dataset richiede tempi di calcolo non trascurabili, ed è stata condotta su infrastrutture con vincoli di tempo variabili, quali sessioni interrotte manualmente o processi terminati per errori esterni al sistema, entrambi gli stadi sono stati progettati per riconoscere, all'avvio, quali record risultino già presenti nel file di output e per procedere unicamente con quelli mancanti. Questa proprietà non è stata prevista fin dalla prima versione del sistema, ma introdotta a seguito di interruzioni ripetute nel corso dello sviluppo, ed è realizzata verificando la presenza di un identificativo di record, o della combinazione tra identificativo e stile linguistico nel caso del secondo stadio, all'interno del file di output già scritto su disco. Una conseguenza diretta di questa proprietà è che un file di output parzialmente scritto, o corrotto da un'interruzione avvenuta durante la scrittura di una singola riga, può impedire la corretta ripresa dell'esecuzione se non viene preventivamente ripulito, poiché il meccanismo di verifica presuppone che ciascuna riga del file sia un record sintatticamente valido.

Il primo stadio è esposto tramite un'interfaccia utente basata su Streamlit, che svolge una duplice funzione architettuale: da un lato consente la generazione interattiva di singoli record o di piccoli insiemi a scopo di verifica, dall'altro espone un meccanismo di generazione batch, invocabile con un singolo comando, che itera sull'insieme completo delle combinazioni di entità, tier e parametri, delegando a

un modulo separato la logica di generazione vera e propria. Questa separazione tra l'interfaccia interattiva e il modulo di generazione batch consente di riutilizzare la medesima logica di generazione sia in un contesto esplorativo, durante lo sviluppo e la verifica del sistema, sia in un contesto di produzione dell'intero dataset, senza duplicazione di codice tra i due utilizzi.

Infine, entrambi gli stadi sono affiancati da un sottosistema di tracciamento statistico, distinto dal meccanismo di validazione descritto nella sezione precedente. Mentre la validazione opera a livello di singolo record, decidendo se una specifica variante debba essere accettata o scartata, il sottosistema di tracciamento aggrega queste decisioni sull'intera esecuzione, mantenendo contatori quali il numero di tentativi di generazione per ciascuno stile linguistico, il numero di accettazioni corrispondenti, e l'elenco dei record per cui nessuna variante valida è stata prodotta entro il numero massimo di tentativi consentito. Questi dati aggregati, persistiti in un file separato dal dataset generato, hanno costituito lo strumento primario per l'identificazione dei problemi sistematici discussi nel capitolo di valutazione, poiché un tasso di accettazione anomalmente basso per uno stile specifico, o per un'entità specifica, ha costituito in più occasioni il primo segnale di un difetto nel sistema di generazione, precedente a qualunque ispezione manuale del testo prodotto.

4.2 Detailed design

Il design del primo stadio è organizzato attorno al sistema a tier introdotto nel capitolo di analisi. Ciascuno dei sei livelli (Tier 0, e Tier da A a E) è associato a un insieme di entità del knowledge graph e a quattro parametri configurabili indipendentemente: il numero minimo e massimo di relazioni attraversate a partire dal nodo radice, il numero minimo e massimo di percorsi alternativi combinati tramite clausole di disgiunzione, il numero minimo e massimo di proprietà vincolate su un medesimo nodo, e il numero minimo e massimo di valori ammessi per ciascun vincolo. La combinazione di questi otto valori, differenziata per ciascun tier, determina l'intervallo di complessità delle query generabili per quel livello.

Il sistema è stato esteso, nel corso del progetto, con un secondo insieme di parametri, distinto dal primo e denominato insieme supplementare, introdotto

per ampliare il volume del dataset senza rigenerare le porzioni già prodotte. La garanzia di non sovrapposizione tra i due insiemi non è affidata a un controllo a posteriori, ma è strutturale: il valore minimo del numero di proprietà vincolate nell'insieme supplementare è impostato esattamente al valore che costituiva il massimo nell'insieme principale, rendendo impossibile, per costruzione, che una query prodotta con l'insieme supplementare condivida lo stesso numero di proprietà di una query già prodotta in precedenza. Una seconda strategia di variazione, complementare a questa, riorganizza le medesime entità in raggruppamenti tematici differenti da quelli del sistema a tier originale, così che anche a parità di parametri di complessità le combinazioni di entità coinvolte in ciascuna query risultino diverse.

Il mascheramento dei valori, primo componente del secondo stadio, sostituisce ciascun valore letterale del prompt con un segnaposto opaco prima di invocare il modello linguistico, e reinserisce il valore originale, o una sua forma alternativa ammessa, dopo la generazione. Questa scelta impedisce al modello di alterare un valore critico durante la parafrasi, poiché il modello non ha accesso al valore stesso durante la generazione. Per le query che coinvolgono un numero elevato di valori, il testo mascherato viene suddiviso in porzioni di dimensione limitata, elaborate separatamente, per ridurre il rischio che un numero eccessivo di segnaposti in un singolo prompt comprometta la qualità della generazione.

Il sistema di stili definisce dieci registri linguistici distinti, applicati ciclicamente alle richieste inviate al modello linguistico, ciascuno caratterizzato da una descrizione testuale del registro atteso fornita al modello come parte del prompt di generazione. Un meccanismo di memoria per stile impedisce che due varianti del medesimo registro condividano un incipit identico o quasi identico, confrontando le prime parole di ciascuna nuova generazione con una finestra delle generazioni più recenti per quello stile. Nel corso dello sviluppo, questo meccanismo si è rivelato inizialmente troppo permissivo: un confronto limitato alle prime quattro parole non catturava ripetizioni percepibili come identiche da un lettore umano, quali varianti che iniziassero tutte con la medesima struttura per le prime due parole ma differissero nella quarta. La correzione ha ristretto il confronto alle prime due parole; questa correzione, tuttavia, ha introdotto un effetto collaterale: per stili con un repertorio lessicale naturale limitato, una finestra di memoria troppo ampia

impediva sistematicamente la generazione di una variante accettabile, causando il fallimento completo di uno slot di generazione. Il sistema è stato corretto una seconda volta, riducendo la finestra di memoria e introducendo un meccanismo di riserva: una variante che supera tutti i controlli di contenuto ma fallisce unicamente il controllo sull'incipit viene comunque accettata se, al termine dei tentativi disponibili, nessuna variante migliore è stata prodotta, evitando così di scartare uno slot intero per un motivo puramente stilistico.

Il sistema di sinonimi affronta la rigidità lessicale che deriverebbe da un uso esclusivo del mascheramento letterale. Il sistema adotta un approccio ibrido a due livelli: per categorie di valori riconoscibili tramite regole deterministiche, quali date, numeri interi di piccola entità, ed etichette composte nella forma "categoria - sottocategoria", una forma alternativa in linguaggio naturale viene generata senza l'intervento di un modello linguistico; per i restanti valori, un modello linguistico viene interrogato per proporre un sinonimo di uso comune. Questa seconda via è soggetta a un filtro di sicurezza che esclude esplicitamente, dal tentativo di generazione del sinonimo, i valori identificabili come codici tramite un prefisso seguito da due punti, i valori di genere anagrafico, e i valori che corrispondono al pattern di una classificazione clinica controllata, quali gradi numerici o codici di stadiazione. Il filtro anti-codice, nella sua prima versione, riconosceva correttamente prefissi interamente alfabetici ma non riconosceva prefissi alfanumerici, lasciando così passare senza esclusione i codici della classificazione ICD-10, il cui prefisso contiene sia lettere sia cifre; la correzione ha esteso il pattern di riconoscimento per includere questo caso.

Il meccanismo di validazione, applicato a ciascuna variante prodotta dal modello linguistico prima della sua accettazione nel dataset finale, verifica due condizioni indipendenti. La prima è la presenza di tutti i valori originali del record, ammettendo sia la forma letterale sia, quando disponibile, la forma alternativa proposta dal sistema di sinonimi. La seconda è la dissimilarità testuale rispetto alle varianti già accettate per il medesimo record, calcolata tramite una soglia di similarità che si adatta alla lunghezza del testo confrontato. Una variante che non supera entrambi i controlli viene scartata e la generazione viene ripetuta; se, dopo un numero massimo di tentativi, nessuna variante valida è stata prodotta, il sistema restituisce il testo mascherato con i valori originali reinseriti nella loro

forma letterale esatta, garantendo comunque la produzione di un record utilizzabile, sebbene privo della variazione linguistica che il processo mira normalmente a introdurre.

Chapter 5

Implementation

5.1 Technology stack

Il primo stadio del sistema è implementato in Python, e la sua interfaccia utente si basa su Streamlit, un framework che consente di costruire applicazioni web interattive direttamente da codice Python, senza la necessità di sviluppare separatamente un front-end. La scelta di Streamlit, discussa dal punto di vista architetturale nel capitolo precedente, è motivata dalla possibilità di esporre, all'interno della medesima applicazione, sia un'interfaccia di generazione interattiva per singoli record, utile durante lo sviluppo e la verifica del sistema, sia un meccanismo di generazione batch invocabile con un singolo comando, riutilizzando la medesima base di codice per entrambi gli utilizzi.

L'endpoint SPARQL del knowledge graph del Cancer Virtual Lab, interrogato dal primo stadio per la verifica preventiva di esistenza dei dati descritta nel capitolo di design, è containerizzato tramite Docker. Questa scelta, non adottata direttamente nell'ambito di questo progetto ma ereditata dall'infrastruttura preesistente del Cancer Virtual Lab, isola l'endpoint e le sue dipendenze dall'ambiente di esecuzione del sistema di generazione, ma ha richiesto, nel corso dello sviluppo, particolare attenzione alla configurazione degli indirizzi di rete utilizzati per raggiungere l'endpoint dall'interno di un ambiente containerizzato, un problema discusso nel dettaglio nella sezione successiva.

La fase di augmentation testuale si basa su Ollama, uno strumento per l'esecuzione

locale di modelli linguistici, che consente di eseguire il modello impiegato senza dipendenza da servizi esterni a pagamento e senza inviare i dati del knowledge graph, di natura clinica, a un servizio di terze parti. Il modello linguistico impiegato per la generazione delle varianti testuali è un modello della famiglia Gemma di dimensioni contenute, eseguito tramite l'interfaccia offerta da Ollama, che gestisce sia il caricamento del modello sia l'esposizione di un'interfaccia programmatica utilizzata dal secondo stadio del sistema per inviare le richieste di generazione.

Per la fase di fine-tuning del modello, distinta dalla fase di augmentation e descritta a livello di obiettivi nel capitolo di analisi, lo stack tecnologico impiegato comprende la libreria Unsloth, che fornisce un'implementazione ottimizzata delle tecniche di fine-tuning a basso numero di parametri addestrabili per modelli della famiglia Gemma, in combinazione con le librerie transformers e trl per la gestione del ciclo di addestramento supervisionato. La scelta di Unsloth rispetto a un'implementazione basata direttamente sulle librerie transformers e peft è motivata dalla disponibilità di ottimizzazioni specifiche per l'architettura del modello impiegato, che riducono il consumo di memoria video durante l'addestramento, un fattore rilevante dato il vincolo sulle risorse di calcolo disponibili discusso nel capitolo di analisi.

L'infrastruttura di calcolo utilizzata è variata nel corso del progetto, in linea con i vincoli discussi nel capitolo di analisi. Le prime fasi di sviluppo sono state condotte su una macchina priva di accelerazione grafica dedicata. Una fase successiva ha fatto uso di una scheda grafica NVIDIA GeForce GTX 1650, con quattro gigabyte di memoria video, sufficiente per la fase di augmentation ma non per il fine-tuning del modello, come verificato empiricamente nel corso dello sviluppo attraverso un tentativo diretto di esecuzione, discusso nella sezione successiva. La fase di fine-tuning richiede infine l'accesso al cluster High Performance Computing (HPC) del DISI, con partizioni dotate di Graphics Processing Unit (GPU) NVIDIA RTX 2080 e L40, la cui allocazione è gestita tramite il sistema di code SLURM, comunemente impiegato in ambienti di calcolo condiviso di questo tipo per coordinare l'accesso a risorse di calcolo limitate tra utenti multipli.

5.2 Implementation highlights

Il meccanismo di ripresa dell'esecuzione, descritto a livello architetturale nel capitolo precedente, è realizzato verificando, all'avvio di ciascuno stadio, quali record risultino già presenti nel file di output esistente. La verifica avviene leggendo l'intero file di output riga per riga all'avvio del processo, e costruendo un insieme degli identificativi già completati, confrontato successivamente con l'insieme completo dei record da generare per determinare quali richiedano ancora una generazione. Per il secondo stadio, la verifica è effettuata sulla combinazione di identificativo del record e stile linguistico, poiché un medesimo record può comparire più volte nel file di output, una volta per ciascuna variante stilistica prodotta, e verificare la sola presenza dell'identificativo del record risulterebbe insufficiente a distinguere un record completato in tutti i suoi stili da uno completato solo parzialmente. Un file separato, distinto dal dataset generato, mantiene un insieme di contatori cumulativi relativi all'intera esecuzione, quali il numero di tentativi e di accettazioni per ciascuno stile linguistico; questo file svolge una funzione puramente diagnostica e non influisce sulla decisione di quali record rigenerare, che dipende unicamente dal contenuto del file di output principale.

Nel corso dello sviluppo si è verificato più volte che un'esecuzione interrotta, per intervento manuale tramite l'invio di un segnale di interruzione al processo, o per un errore esterno al sistema quale la scadenza del tempo assegnato su un'infrastruttura di calcolo condivisa, lasciasse il file di output con l'ultima riga scritta solo parzialmente, corrispondente a un oggetto in formato JSON incompleto. Poiché il meccanismo di ripresa presuppone che ciascuna riga del file sia sintatticamente valida e analizzabile singolarmente, questa condizione impediva l'avvio di qualunque esecuzione successiva, con un errore di analisi del formato dati sollevato all'atto della lettura della prima riga non valida incontrata, spesso non la primissima riga del file ma una riga intermedia corrispondente al punto esatto in cui l'esecuzione precedente era stata interrotta. La diagnosi di questo problema, inizialmente non ovvia poiché il messaggio di errore riportato dalla libreria di analisi del formato JSON non indica esplicitamente quale riga del file ne sia la causa, ha richiesto lo sviluppo di uno script indipendente dal sistema principale, in grado di analizzare il file riga per riga, tentare l'analisi di ciascuna

singolarmente, riconoscere le righe non valide segnalandone il numero, e produrre una versione ripulita del file scartando unicamente le righe corrotte, preservando tutte le altre senza necessità di rigenerarle.

La modalità alternativa di generazione, che omette il mascheramento dei valori, è stata introdotta per condurre un confronto sperimentale sulla qualità strutturale del testo prodotto, motivato dall'ipotesi che l'assenza di segnaposti, sostituiti nel testo con i valori originali accompagnati da suggerimenti di forma alternativa come indicazioni facoltative, potesse produrre frasi con una struttura sintattica più naturale rispetto alla modalità principale, in cui il modello linguistico compone la frase attorno a segnaposti privi di significato lessicale. Il confronto è stato condotto su un insieme di venti record di input, selezionati per rappresentare una gamma di complessità crescente, con i medesimi parametri di generazione applicati a entrambe le modalità. I risultati, riportati nel dettaglio nel capitolo di valutazione, hanno mostrato un tasso di accettazione complessivo lievemente superiore nella modalità con mascheramento, e un comportamento distinto nella natura dei fallimenti osservati tra le due modalità.

L'infrastruttura di calcolo impiegata per l'esecuzione del secondo stadio è cambiata più volte nel corso del progetto, in un percorso che ha seguito da vicino la disponibilità progressiva di risorse hardware. Nella fase iniziale, l'esecuzione è avvenuta su una macchina priva di accelerazione grafica dedicata, con tempi di generazione dell'ordine dei minuti per singola variante, un ordine di grandezza che ha vincolato questa fase a test su campioni ridotti piuttosto che alla produzione del dataset completo. In una fase successiva, è stata resa disponibile una scheda grafica NVIDIA GeForce GTX 1650, con quattro gigabyte di memoria video. L'utilizzo di questa scheda ha inizialmente incontrato un problema pratico non previsto: la cartella di installazione predefinita dei modelli linguistici scaricati da Ollama, collocata sul disco di sistema, ha esaurito lo spazio disponibile durante lo scaricamento del modello impiegato, un problema risolto reindirizzando tale percorso, tramite una variabile d'ambiente dedicata, su un volume con maggiore spazio disponibile. Un tentativo di accelerazione su una seconda macchina, dotata di scheda grafica integrata Intel anziché di scheda dedicata, condotto per verificare se l'accelerazione grafica fosse disponibile anche su questo hardware meno performante, ha permesso di verificare che lo strumento di esecuzione locale del mod-

ello supporta nativamente l’accelerazione tramite l’interfaccia Vulkan su schede grafiche di questo tipo, contrariamente a un’aspettativa iniziale basata sulla documentazione di progetti di terze parti, nel frattempo non più mantenuti, dedicati specificamente all’accelerazione di modelli linguistici su schede grafiche integrate Intel. La fase di fine-tuning del modello, per la quale la memoria disponibile sulla scheda grafica consumer risulta insufficiente secondo quanto verificato empiricamente tramite un tentativo diretto di esecuzione, richiede infine l’accesso al cluster di calcolo condiviso del dipartimento, con l’esecuzione gestita tramite un sistema di code e la possibilità di ripresa automatica in caso di interruzione per scadenza del tempo assegnato, realizzata applicando al processo di fine-tuning il medesimo principio di verifica dello stato già completato descritto all’inizio di questa sezione per il sistema di generazione del dataset.

Nel corso dello sviluppo sono stati identificati e corretti diversi problemi tecnici, distinti da quelli relativi alla qualità linguistica del testo generato, discussi nel capitolo di valutazione, e riconducibili invece a incongruenze nell’integrazione tra i diversi componenti del sistema. Un’incongruenza nell’interfaccia dell’endpoint SPARQL ha causato il fallimento sistematico di una funzionalità di conteggio delle istanze corrispondenti a una combinazione di criteri, poiché il codice del primo stadio invocava una rotta dell’interfaccia di programmazione non più disponibile; il problema è stato risolto individuando, tramite ispezione delle rotte effettivamente esposte dall’interfaccia, quella corretta per la medesima funzionalità, e adattando il formato della risposta attesa dal codice, poiché la nuova rotta restituisce i dati secondo una struttura diversa dalla precedente. Un problema di configurazione dell’interfaccia interattiva, dovuto al fatto che l’ambiente di esecuzione di Streamlit non aggiunge automaticamente al percorso di ricerca dei moduli Python le cartelle contenenti pagine secondarie dell’applicazione, ha impedito l’importazione del modulo responsabile della generazione batch quando invocato dall’interfaccia grafica, pur funzionando correttamente se invocato direttamente da riga di comando; il problema è stato risolto aggiungendo esplicitamente, all’interno del gestore dell’evento associato al pulsante di generazione batch, il percorso necessario al percorso di ricerca dei moduli. Un indirizzo di rete impostato in modo statico nel codice, corrispondente all’indirizzo dell’endpoint SPARQL raggiungibile dalla macchina di sviluppo ma non dall’interno dell’ambiente containerizzato

in cui l'applicazione Streamlit viene eseguita, ha impedito il funzionamento della generazione batch quando invocata dall'interno di tale ambiente, fino alla sua sostituzione con un parametro configurabile, valorizzato riutilizzando la medesima configurazione di indirizzo già correttamente impiegata altrove nell'applicazione per le medesime richieste effettuate dall'interfaccia interattiva.

Chapter 6

Evaluation

6.1 Methodology

La valutazione del dataset prodotto si è basata su due approcci complementari, applicati congiuntamente e non in alternativa, e ripetuti in forma ciclica nel corso dello sviluppo anziché applicati una sola volta al termine della generazione: ciascun problema individuato ha motivato una correzione al sistema di generazione, seguita da una nuova applicazione di entrambi gli approcci sul dataset corretto, in un processo iterativo descritto con maggiore dettaglio nella sezione successiva per ciascun problema specifico.

Il primo approccio, di natura programmatica, verifica automaticamente proprietà strutturali di ciascun record, ciascuna delle quali corrisponde a una condizione formalmente verificabile senza intervento umano. La corrispondenza esatta tra il testo del prompt e la query SPARQL associata verifica che nessuna variante sia stata accidentalmente associata a una query diversa da quella originaria, un controllo di integrità basilare ma non garantito automaticamente dal sistema di generazione batch. L'assenza di prompt duplicati all'interno del medesimo insieme di dati verifica che il meccanismo di validazione della dissimilarità testuale, descritto nel capitolo di design, abbia effettivamente impedito la produzione di due varianti testualmente identiche per il medesimo record. La presenza di tutti i valori originali nella loro forma letterale o in una forma alternativa ammessa verifica il rispetto del requisito di tracciabilità discusso nel capitolo di analisi. L'assenza

di sintassi propria di un linguaggio di interrogazione strutturato riprodotta letteralmente nel testo in linguaggio naturale, e l'assenza di parentesi non riconducibili a un valore letterale originale, verificano invece proprietà di naturalezza del testo prodotto, introdotte a seguito dell'individuazione dei problemi discussi nella sezione successiva, e successivamente mantenute come controlli permanenti applicati a ogni nuova generazione.

Il secondo approccio, di natura qualitativa, consiste nella lettura diretta di campioni casuali del dataset, ripetuta su estrazioni indipendenti multiple nel corso dello sviluppo, tipicamente comprendenti alcune decine di varianti selezionate casualmente dall'insieme completo disponibile in quel momento. A differenza del controllo programmatico, applicato in modo automatico e sistematico sull'intero dataset, il controllo qualitativo richiede la lettura diretta del testo da parte di chi conduce la valutazione, ed è stato pertanto applicato a campioni di dimensione limitata anziché sull'intero dataset in ciascuna occasione; la ripetizione su estrazioni indipendenti multiple, condotta in momenti diversi dello sviluppo, ha in parte compensato questa limitazione di copertura.

I due approcci non sono ridondanti, ma catturano categorie di problemi distinte, e questa distinzione si è rivelata concretamente rilevante nel corso della valutazione. Il controllo programmatico individua in modo affidabile violazioni di proprietà formalmente definibili, quali la presenza di un valore o l'assenza di duplicati, ma non è in grado di riconoscere un'alterazione di significato che preservi comunque la struttura superficiale del testo, poiché una condizione di questo tipo non corrisponde a una regola formalmente esprimibile senza incorporare conoscenza del dominio clinico specifico. Il controllo qualitativo, al contrario, ha permesso di individuare problemi di questo tipo, quali la sostituzione di un termine clinico controllato con un termine di registro comune concettualmente distinto, che nessuno dei controlli programmatici disponibili in quel momento era in grado di rilevare per costruzione. In più occasioni, un problema individuato per la prima volta tramite il controllo qualitativo ha successivamente motivato l'introduzione di un nuovo controllo programmatico mirato, in grado di quantificare la portata esatta del problema sull'intero dataset, un pattern ricorrente descritto nel dettaglio per ciascun caso nella sezione successiva.

6.2 Quality issues found

Nel corso della valutazione sono stati individuati diversi problemi di qualità, di natura e portata differenti, ciascuno diagnosticato fino alla causa tecnica sottostante e corretto nel sistema di generazione. La presentazione che segue riporta, per ciascun problema, non solo la causa individuata ma anche il percorso diagnostico seguito, poiché in più occasioni tale percorso si è rivelato replicabile per problemi successivi di natura simile.

Il primo problema individuato durante la lettura a campione del dataset ha riguardato la confusione tra l'acronimo clinico ECOG e il termine ecocardiogramma. Il record interessato conteneva, tra le sue varianti stilistiche generate, tre formulazioni che facevano riferimento correttamente allo stato funzionale ECOG del paziente, e una quarta variante che sostituiva l'espressione con un termine che unisce erroneamente un esame diagnostico cardiologico a una scala priva di relazione con la funzione cardiaca. La scoperta di questo caso non è avvenuta tramite il controllo programmatico, che verifica unicamente la presenza dei valori vincolati dalla query e non il contenuto testuale circostante, ma tramite lettura diretta comparativa delle quattro varianti dello stesso record affiancate, un metodo di ispezione rivelatosi più volte capace di individuare errori invisibili ai controlli automatici. Poiché il termine originale non corrisponde a un pattern di codice riconoscibile dal filtro di sicurezza del sistema di sinonimi, il caso è stato classificato come un'allucinazione isolata del modello linguistico durante la generazione libera della frase, non riconducibile a un difetto specifico e correggibile del sistema.

Il secondo problema, di portata quantitativamente maggiore, è stato scoperto in circostanze simili: un confronto diretto tra le varianti stilistiche di un medesimo record, relativo questa volta a due codici distinti della classificazione ICD-10. Osservando le cinque varianti generate per quel record, alcune preservavano entrambi i codici nella loro forma letterale, mentre altre li sostituivano con espressioni testuali di senso compiuto ma diverse tra loro e reciprocamente incoerenti, indizio decisivo che si trattasse di un'allucinazione del modello linguistico e non di una traduzione plausibile sebbene imprecisa. L'indagine della causa tecnica ha richiesto l'ispezione diretta del filtro di sicurezza del sistema di sinonimi, rivelando che il filtro riconosceva correttamente prefissi di codice composti interamente da lettere,

ma non riconosceva il prefisso della classificazione ICD-10, poiché quest'ultimo contiene una cifra, non prevista dal pattern originale limitato ai soli caratteri alfabetici. Una verifica sistematica condotta successivamente sull'intero dataset, confrontando per ciascun identificativo di record il numero di occorrenze letterali del prefisso tra le sue diverse varianti stilistiche, ha permesso di quantificare la portata del problema in quarantasette record complessivamente coinvolti.

Il terzo problema è emerso non dalla lettura di un singolo record, ma da una ricerca mirata condotta sull'intero dataset dopo aver notato, in un campione casuale, un numero sproporzionato di occorrenze del termine "Woman" in contesti dove il valore originale della query era "Female". Una ricerca sistematica di questo pattern sull'intero dataset ha confermato che il fenomeno non era isolato: duecentonove varianti, corrispondenti a circa il sette per cento del totale, presentavano la sostituzione "Female" con "Woman" o, analogamente, "Male" con "Man". A differenza dei due problemi precedenti, qui il rischio non risiedeva nella plausibilità della sostituzione, lessicalmente ineccepibile, ma nel suo significato: il campo del knowledge graph da cui il valore proviene è denominato esplicitamente come genere inteso quale sesso assegnato alla nascita, una distinzione concettuale deliberata rispetto all'identità di genere. La causa tecnica è risultata la medesima categoria di difetto del problema precedente: il filtro di sicurezza valutava un candidato per la generazione di un sinonimo unicamente sulla base di criteri sintattici, senza consapevolezza semantica del dominio clinico a cui il valore apparteneva.

Il quarto problema, individuato nel medesimo giro di ispezione mirata che aveva rivelato il terzo, ha riguardato la sostituzione di classificazioni cliniche numeriche, quali "Grade 5", con espressioni tratte da un dominio concettualmente adiacente ma distinto, quali "Fifth grade", termine associato in inglese comune a un livello scolastico anziché a una scala clinica. La ricerca è stata condotta estendendo il medesimo approccio impiegato per il terzo problema a un pattern testuale più generale, corrispondente a un numero ordinale scritto per esteso seguito dalla parola "grade"; questa ricerca ha individuato tredici casi, confermando che il filtro sintattico privo di consapevolezza semantica non proteggeva unicamente i valori anagrafici ma qualunque valore numerico associato a una classificazione clinica espressa nella forma "categoria numero".

Il quinto problema, di natura diversa dai precedenti poiché riscontrato non

nell'output del sistema automatico ma nella prima fase del lavoro di generazione manuale delle varianti per gli insiemi di validazione e di test, ha riguardato la produzione di testo non conforme a un registro di linguaggio naturale autentico. In particolare, alcuni modelli di frase impiegati per lo stile denominato "tecnico" riproducevano letteralmente sintassi propria di un linguaggio di interrogazione strutturato, mentre altri modelli, per stili concepiti come sintetici, si limitavano a un elenco di coppie campo-valore separate da virgole, senza costituire una frase di senso compiuto in linguaggio naturale; in entrambi i casi, il problema violava esplicitamente la definizione data allo stile stesso, la cui descrizione testuale forniva al modello indicazioni per evitare precisamente questo tipo di formulazione. Il problema è stato individuato tramite lettura diretta e sistematica dell'insieme completo delle varianti prodotte manualmente, non tramite campionamento, data la dimensione contenuta di questa porzione del lavoro rispetto al dataset generato automaticamente. La correzione ha richiesto la riscrittura completa dei modelli di frase per gli stili interessati, seguita da una verifica sistematica dell'assenza delle forme problematiche sull'insieme rivisto.

Il sesto problema ha riguardato la concatenazione di due valori data adiacenti in un intervallo temporale, priva di un separatore testuale intermedio, risultante in una sequenza di cifre e trattini non interpretabile senza ambiguità da un lettore umano, poiché le date stesse contengono trattini come parte della propria formattazione interna. Il problema è emerso durante la lettura di varianti particolarmente lunghe, contenenti più intervalli temporali distinti nella medesima frase, dove la concatenazione risultava visivamente evidente. Una ricerca sistematica basata su un'espressione regolare, costruita per riconoscere due sequenze nel formato di una data ISO adiacenti separate unicamente da un trattino, ha individuato cinquantacinque occorrenze del problema nell'insieme allora disponibile. La correzione, applicata direttamente come trasformazione testuale sull'output già generato anziché richiedere una nuova generazione, ha inserito sistematicamente una congiunzione testuale tra i due valori data in ciascuna occorrenza individuata.

Il settimo problema ha riguardato l'uso di una preposizione non idonea, "at", in combinazione con valori riferiti a una linea di trattamento terapeutico, dove la preposizione corretta in inglese è "on". La ricerca sistematica di questo pattern ha inizialmente prodotto un numero di corrispondenze superiore a quello poi

confermato, poiché il medesimo pattern testuale intercettava anche casi in cui la preposizione "at" compariva legittimamente in combinazione con un valore di natura geografica o anatomica, quale la sede di una diagnosi; la distinzione tra i due casi ha richiesto una verifica manuale di ciascuna corrispondenza individuata dalla ricerca automatica, riducendo il conteggio finale a otto casi genuini.

Un ottavo problema, distinto dai precedenti poiché non riconducibile a un difetto del sistema di generazione delle varianti, ha riguardato la presenza del valore letterale "nan" come unità di misura in un numero limitato di record, riscontrato incidentalmente durante una delle ricerche sistematiche condotte per altri problemi. Questo valore, tipico di una rappresentazione di dato mancante non gestita a monte nella fase di costruzione del dataset strutturato, è stato preservato correttamente dal sistema di augmentation, che per costruzione non altera i valori letterali originali indipendentemente dal loro contenuto; il problema, riscontrato in sette record, è stato pertanto segnalato come appartenente alla fase di generazione del dataset strutturato, non alla fase di augmentation linguistica, e non è stato corretto nell'ambito di questo lavoro.

Il nono problema, di natura strutturale e non linguistica, ha riguardato la presenza, all'interno di una medesima query, di due vincoli distinti sulla medesima proprietà scalare del paziente, quale la data di decesso, con valori incompatibili tra loro, risultanti in una condizione logicamente non soddisfacibile da alcun paziente reale. La scoperta di questo problema, precedente cronologicamente a tutti gli altri qui descritti, è avvenuta durante la lettura di una singola query generata, in cui la stessa proprietà compariva due volte con intervalli di valori disgiunti. La causa è stata ricondotta a un comportamento del generatore di query strutturate, in cui due percorsi relazionali paralleli, campionati indipendentemente durante la costruzione della query, possono in alcuni casi ricampionare la medesima proprietà diretta del nodo paziente con valori differenti. Il problema è risultato marcatamente più frequente nel secondo passaggio di generazione del dataset, introdotto per ampliare la copertura senza sovrapporsi al primo passaggio, che impone un numero minimo di proprietà vincolate per record più elevato rispetto al primo: una verifica sistematica condotta su tale passaggio ha individuato duecentonovantacinque record coinvolti su milletrecentotrentuno esaminati, corrispondenti a circa il ventidue per cento del totale, una proporzione sostanzialmente più alta di quanto

osservato nel dataset prodotto dal primo passaggio di generazione, coerente con l'ipotesi che un numero minimo di proprietà più elevato aumenti proporzionalmente la probabilità di una collisione di questo tipo. La mitigazione adottata ha escluso i record interessati dal dataset finale tramite lo stesso controllo programmatico impiegato per la diagnosi; una correzione alla radice del meccanismo di campionamento dei percorsi paralleli, che richiederebbe una modifica al generatore di query strutturate discusso nel capitolo di design, non è stata implementata nell'ambito di questo lavoro ed è discussa come sviluppo futuro nel capitolo conclusivo.

6.3 Comparison between masking and no-masking generation

Al fine di verificare se l'assenza di mascheramento dei valori producesse una struttura linguistica delle varianti generate qualitativamente diversa rispetto alla modalità principale, è stato condotto un confronto sperimentale su un insieme di venti record di input, selezionati per rappresentare una gamma di complessità crescente, dal caso di un singolo valore vincolato fino al caso di dodici valori vincolati nella medesima query. Le due modalità sono state eseguite sul medesimo insieme di record, con i medesimi parametri di generazione.

Il numero complessivo di slot di generazione tentati è risultato identico tra le due modalità, sessantasei, poiché determinato dalla medesima formula adattiva basata sul numero di valori per record, indipendente dalla modalità di mascheramento. Il tasso di successo complessivo è risultato lievemente superiore nella modalità con mascheramento, novantotto virgola cinque percento contro novantasette percento nella modalità senza mascheramento. La causa dei fallimenti si è rivelata inoltre di natura diversa tra le due modalità: nella modalità senza mascheramento, i fallimenti osservati sul medesimo record sono stati causati da una corrispondenza testuale identica al cento percento tra tentativi di generazione distinti, condizione non osservata nella modalità con mascheramento, dove il fallimento osservato ha mostrato una corrispondenza testuale parziale, inferiore al novanta percento.

Questo risultato suggerisce che l'assenza di segnaposti opachi, pur non com-

promettendo in modo sostanziale il tasso di successo complessivo, può ridurre la capacità del modello linguistico di produrre formulazioni distinte quando il testo con i valori visibili offre un numero limitato di modi naturali per esprimere la medesima richiesta.

6.4 Dataset statistics

Il dataset finale risulta suddiviso in tre insiemi distinti, destinati rispettivamente all'addestramento, alla validazione, e al test del modello. L'insieme di addestramento comprende [NUMERO] varianti, prodotte a partire da [NUMERO] record sorgente distinti, con una media di [NUMERO] varianti per record. Gli insiemi di validazione e di test comprendono rispettivamente [NUMERO] e [NUMERO] varianti.

La distribuzione delle varianti tra i dieci registri stilistici risulta [uniforme / non uniforme in favore di NOME STILE], con lo stile [NOME] rappresentato in [NUMERO] varianti e lo stile meno rappresentato, [NOME], in [NUMERO] varianti.

Una differenza rilevante nella densità di parafrasi si osserva tra l'insieme di addestramento, prodotto tramite la formula adattiva basata sul numero di valori per record, e gli insiemi di validazione e di test, prodotti con un numero fisso di cinque varianti per record: il primo presenta una media di circa [NUMERO] varianti per record, contro le cinque fisse degli altri due insiemi.

Chapter 7

Conclusion and Future Work

Bibliography

- [DBAL19] Mohnish Dubey, Debayan Banerjee, Abdelrahman Abdelkawi, and Jens Lehmann. Lc-quad 2.0: A large dataset for complex question answering over wikidata and dbpedia. In *The Semantic Web – ISWC 2019*, pages 69–78. Springer, 2019.
- [LdCF25] A. B. Labate, B. W. S. R. de Carvalho, and S. R. Fiorini. Synthetic dataset generation and validation with LLMs for text2sparql in Wikidata graphs. In *Proceedings of the Wikidata Workshop 2025*, 2025.
- [SCKS20] B. A. Spiegel, V. Cheong, J. E. Kaplan, and A. Sanchez. MK-SQuIT: Synthesizing questions using iterative template-filling, 2020.
- [TMDL17] Priyansh Trivedi, Gaurav Maheshwari, Mohnish Dubey, and Jens Lehmann. Lc-quad: A corpus for complex question answering over knowledge graphs. In *The Semantic Web – ISWC 2017*, pages 210–218. Springer, 2017.

Acknowledgements